

Playing the Game of Snake with Limited Knowledge: Unsupervised Neuro-Controllers Trained Using Particle Swarm Optimization

Cornelius J. van Rooyen
Department of Computer Science
University of Pretoria
Pretoria, Gauteng, South Africa
e-mail: nvanrooyen@cs.up.ac.za

Willem S. van Heerden
Department of Computer Science
University of Pretoria
Pretoria, Gauteng, South Africa
e-mail: wvheerden@cs.up.ac.za

Christopher W. Cleghorn
Department of Computer Science
University of Pretoria
Pretoria, Gauteng, South Africa
Email: ccleghorn@cs.up.ac.za

Abstract—Methods in the domain of artificial intelligence (AI) have been applied to develop agents capable of playing a variety of games. The single-player variant of Snake is a well-known and popular video game that requires a player to navigate a line-based representation of a snake through a two-dimensional playing area, while avoiding collisions with the walls of the playing area and the body of the snake itself. A score and the snake length are increased whenever the snake is moved through items representing food. The game thus becomes more challenging as the score increases. The application of AI techniques to playing the game of Snake has not been very well explored. This paper proposes a novel technique that uses particle swarm optimization for the unsupervised training of neuro-controllers in order to play the game of Snake. The proposed technique assumes nothing about effective game playing strategies, and thus works with limited knowledge. Sensory input is also minimal. Due to the lack of similar AI-based approaches for playing Snake, the proposed technique is empirically compared against three hand-designed Snake playing agents in terms of several performance measures. The performance of the proposed technique demonstrates the feasibility of the approach, and suggests that future research into AI-based controllers for Snake will be fruitful.

Keywords—Artificial intelligence, neuro-controllers, particle swarm optimization, agents, game, Snake.

I. INTRODUCTION

The field of artificial intelligence (AI) is broad and involves many different sub-disciplines [1]. Many AI paradigms have been applied to a variety of different games [2] since the earliest years of AI research. AI approaches are capable of successfully playing games as varied as Checkers [3], Chess [4], and Go [5]. An algorithm that plays a game is called an agent. There are many ways to build agents, ranging from manually programmed and hand-optimized expert systems to methods based on advanced machine learning paradigms.

The game of Snake falls within a class of video games first seen in the early days of computing [6]. While some research on the single-player variant of the game exists, little work has used adaptive AI approaches to learn playing strategies.

The primary contribution of this research is the introduction of a novel unsupervised approach that uses particle swarm optimization (PSO) to train neuro-controllers in the playing of Snake. The technique assumes nothing about optimal or safe playing strategies, thus working with limited knowledge. Addi-

tionally, minimal sensory input is provided. Due to the lack of similar methods, three hand-designed agents were empirically compared to the proposed method using several performance measures. This research is a feasibility demonstration for the new method, not a detailed performance investigation.

The rest of this paper is organized as follows: Section II outlines the game of Snake, while Section III describes the novel technique proposed by this work. Section IV presents the results of the empirical analysis, after which Section V summarizes conclusions and possible future research avenues.

II. THE GAME OF SNAKE

This section covers the game of Snake. Section II-A describes the general characteristics of the game, while Section II-B surveys previous research done on the game.

A. Characteristics of the Game

A line represents a snake within a usually two-dimensional playing area. The snake moves in the direction its head faces, and the body follows the path of the head. The player changes the direction of snake movement, thus navigating through the playing area. The game ends when the snake collides with the edge of the playing area, another obstacle, or the snake body itself. Two categories of Snake-like games exist.

- 1) Some games only have a single player, and the objective is to gain as high a score as possible before a collision. Because the playing area is a finite size and the snake never decreases in length, the game is guaranteed to end eventually. This type of game was popularized through inclusion with early Nokia mobile devices [7].
- 2) Other Snake variants have two players, each controlling a snake in a shared playing area. A player who navigates into a collision loses, and each player attempts to encircle and block their opponent in order to instigate a collision. Blockade [6], developed in 1976, is the earliest Snake-like game and falls into this category.

Usually the snake lengthens over time to increase game difficulty. Games in the first category usually randomly add food items to the playing area. When the snake's head moves over a food item, a score increases and the body lengthens.

B. Previous Research on the Game of Snake

A body of research exists for the second category of Snake games [8]–[10], while the first has received far less attention. This is likely due to the existence of simple movement patterns that eventually attain perfect scores [11]. However, such patterns are inefficient and need expert human knowledge, characteristics that the proposed method aims to avoid.

All AI-based game playing agents base decisions on environmental characteristics called features. How well an agent plays a game is defined by a measure of agent fitness.

Ehlis proposes three methods that use genetic programming (GP) to train Snake-playing agents [11]. All three approaches evolve tree-based controllers with leaf nodes representing movement options for the snake. The approaches differ in terms of the following characteristics.

- 1) The first method considers four features: whether food is in front of the head and whether obstacles appear directly in front, to the left, and to the right of the head. Solution fitness is the number of extra points needed for a perfect score after one game, penalized by the distance between the snake and food if no food was eaten.
- 2) The second method uses the features of the first, and includes seven more: whether an obstacle is two cells in front of the head, whether food is above the head, whether food is right of the head, and whether the snake is moving right, left, up, or down. Fitness is measured in the same way as described for the first method.
- 3) The final method operates in the same way as the second method does. However, if a perfect solution is not found by the end of training or the population stagnates for many iterations, the best controller that has been found is included in the initial population of a new GP, for which fitness is averaged over four games.

These approaches base movement decisions on environmental features that differ from those used by the novel approach, and are thus not considered in the experimental analysis.

Lockhart [12] proposes variants of a reinforcement learning (RL) method for training a Snake playing agent. Food consumption is rewarded, while collisions and long periods without food consumption are punished. A value function predicts the agent's future reward using TD(λ) [13], a temporal difference learning technique. An action policy dictates the agent's movement decisions, and is learned using an actor-critic architecture. The following feature set alternatives exist.

- 1) The states the cells in either a 5×5 or a 7×7 grid centered on the head. Cells containing obstacles, food, or nothing receive values of -1 , 1 , and 0 , respectively.
- 2) Whether there is an obstacle one cell ahead, left, and right of the head, and whether food is anywhere ahead, left, or right of the head with no obstacle in the way.
- 3) The features under point 2, including whether there is an obstacle two cells ahead, left, or right of the head, and whether the agent is moving up, down, left, or right.

These approaches are not analyzed as they are not unsupervised and use features not considered by the novel method.

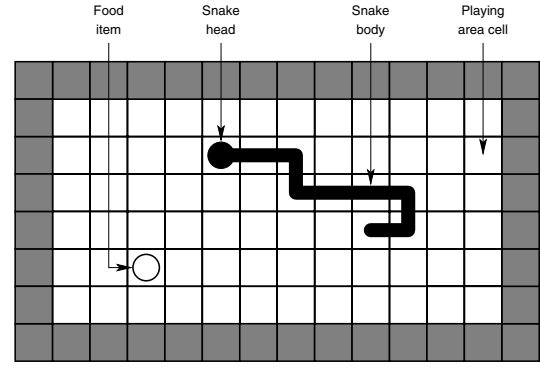


Fig. 1. General characteristics of the implemented Snake game.

An evolutionary approach has also been proposed for training agents to play a modified version of the Snake game in which some food items reduce the score and the length of the snake [14]. A weighted combination of rating functions scores each possible next move, and the highest scoring move is executed. The approach uses four rating functions.

- 1) The smoothness function finds, for each cell, the smallest number of moves needed to reach the cell, and takes on the largest of these values. This estimates how far the snake can move while limiting direction changes.
- 2) The space function evaluates to the number of playing area cells that are reachable from the snake's current position. A larger value indicates a position that provides a greater scope for safe future movement.
- 3) Two functions estimate the possibilities of reaching each food type safely. An unreachable food type gives a value of zero. A reachable food type produces a value of the space function divided by the distance to the food.

A real-valued genetic algorithm (GA) evolves two weights for each rating function: one affecting the importance of the rating function, and another affecting how this importance changes as snake length increases. The changed nature of the game and the complexity of the considered features preclude the inclusion of this method in the experimental analysis.

III. THE PROPOSED TECHNIQUE

This section presents the novel technique proposed by this research. Section III-A describes the implementation details of the Snake game that was used. The two components of the approach, neuro-controllers and particle swarm optimization, are discussed separately in Sections III-B and III-C.

A. Snake Game Implementation Details

The game implementation used by the proposed technique belongs to the first Snake game category mentioned in Section II-A. The important aspects of the game implementation are shown in Fig. 1. The playing area is a grid, within which the snake head and body segments each occupy unique cells. Surrounding the playing area is a boundary of obstacles. To simplify the problem, no other obstacles are introduced. The

Feature	Value	Meaning
1 to 8	1	Cell in (b) corresponding to feature number is a food item
	0.5	Cell in (b) corresponding to feature number is unoccupied
	-0.5	Cell in (b) corresponding to feature number is a body segment
	-1	Cell in (b) corresponding to feature number is a wall
9	1	Food is positioned in a column right of the snake head
	0	Food is positioned in the same column as the snake head
	-1	Food is positioned in a column left of the snake head
10	1	Food is positioned in a row above the snake head
	0	Food is positioned in the same row as the snake head
	-1	Food is positioned in a row below the snake head

(a)

1	2	3
8	●	4
7	6	5

(b)

Fig. 2. Environmental features for PSO trained neuro-controllers. (a) Feature value meanings. (b) Cell states around snake head perceived by neuro-controllers.

snake initially consists of only a head in the middle of the grid. One food item is located in a random grid cell.

In each game step the snake must be moved either up, down, left, or right. Unlike some game implementations, it is thus possible for the snake to move backwards into its own tail. Movement into an unoccupied cell relocates the head to that cell, and removes the last cell in the body. The game ends if the snake attempts to move into the edge of the playing area or into a cell occupied by any part of the snake itself.

As soon as the snake head moves into a cell occupied by a food item, the food item is removed from the playing area, a new food item is placed in a random game area cell, and the last cell in the body is not removed for the following move (thus increasing the snake length by one cell). The game is won when the snake covers all cells in the playing area.

B. Neuro-controllers

The neuro-controllers used by the proposed technique require sensory input from the game environment. Excess features increase the complexity of the problem faced by both the neuro-controller and PSO components of the proposed approach. To reduce complexity, the proposed method uses a minimal set of only ten features, which is shown in Fig. 2 (a). The first eight features perceive the state of the cells neighboring the snake head, which are shown in Fig 2 (b).

Neuro-controllers are artificial neural networks (ANNs) that generate control outputs for agents from environmental inputs. An ANN [15] consists of neurons, each of which receives real-valued inputs that are combined into a net input signal, denoted as *net*. The proposed approach computes *net* as follows:

$$net = \sum_{i=1}^I s_i w_i \quad (1)$$

where I is the number of inputs, s_i represents the i th input, and w_i is a real-valued weight associated with s_i .

An activation function, $f(net)$, computes a neuron's output from the *net* signal. All neurons use the hyperbolic tangent activation function. Feature values must be in the active range of the hyperbolic tangent function, and are therefore all chosen to range between 1 and -1 to satisfy this requirement.

A feedforward neural network architecture is used, where neurons are arranged into an input layer, followed by a hidden layer, and finally an output layer. Layers are fully connected, so that every neuron output in a layer is sent to all the neurons in the next layer. Each input neuron receives one of the features outlined in Fig. 2 (a), and each output neuron represents one of the four possible agent movement directions.

The input and hidden layers each contain one bias unit that produces a constant output of -1. The outputs of a layer's bias unit, combined with weight values, simulate adaptive thresholds on *net* for the neurons in the next layer.

The input signals are propagated from the input neurons, through the hidden neurons, and to the output neurons. The output signals of the final layer each correlated to a movement direction for the snake head. The movement direction with the largest output signal is used as the next agent action.

Fig. 3 illustrates the basic architecture of a neuro-controller in the proposed approach. There are m features and o agent actions. An arbitrary number of hidden neurons is possible.

C. Particle Swarm Optimization

The PSO algorithm [16] defines a swarm of particles, each represented by a v -dimensional solution vector of continuous values defining the position of the particle in a search space. The proposed approach associates one particle dimension with each weight in a neuro-controller. A swarm of particles is initialized with random weights in the range

$$\left[\frac{-1}{\sqrt{fanin}}, \frac{1}{\sqrt{fanin}} \right] \quad (2)$$

where $fanin$ is the number of input neurons when input-to-hidden weights are initialized, and the number of hidden neurons when hidden-to-output weights are initialized, where the bias units are included in each count. This initialization range is proposed by Wessels and Barnard [17].

The standard *gbest* PSO [16] is used, which requires the definition of particle fitness. A particle's fitness is the mean score achieved by the associated neuro-controller over 100 games, where each game is limited to 1 000 moves. The PSO inertia weight [18], denoted ξ , is also included to balance the

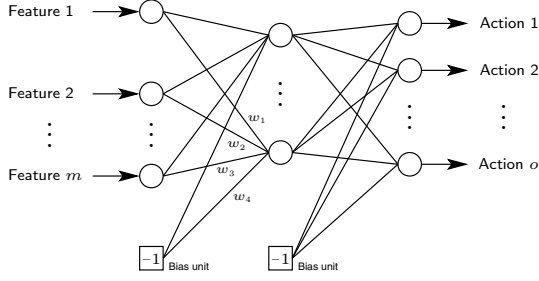


Fig. 3. Basic architecture of a neuro-controller.

exploration and exploitation of the swarm. PSO training ceased after 1000 iterations, or when the fittest particle achieved a perfect score over all games it was evaluated on.

IV. EXPERIMENTAL WORK

This section describes the experimental analysis that was conducted. Section IV-A details the analysis procedure, and Section IV-B presents and interprets the experimental results.

A. Experimental Procedure

In the reported experimental work, the PSO trained neuro-controller was implemented as described as in Section III. The approach was configured with the parameters in Table I. The PSO parameters are those suggested by Clerc and Kennedy [19], which have been shown to perform well.

Three simple hand-optimized agents were designed for comparison against the approach proposed by this research. The first agent (A_1) moves in a random direction at each game step, avoiding moves that instantly end the game unless no safe move is available. This agent allows confirmation that the proposed approach performs better than chance.

The second agent (A_2) moves away from high densities of nearby obstacles while moving towards food. Based on Fig. 2 (b), the density value is computed for cells 2, 4, 6, and 8. For cell 2 the density is the number of obstacles in cells 1, 2, and 3. The density for cell 4 is the obstacle count in cells 3, 4, and 5. Cells 5, 6, and 7 define the density for cell 6. The density of cell 8 is defined by cells 1, 7, and 8. The density values of cells that will move the agent closer to food are decremented to bias agent movement towards food. The agent moves in the direction of the non-obstacle cell with the lowest density value, and breaks ties randomly.

The third agent (A_3) aims to skirt around obstacles. The feature evaluation of this agent is inspired by a simple wall-following agent that was proposed by Nilsson [20]. The control algorithm for the third agent is shown in Fig. 4.

The attention of the reader is drawn to the fact that none of the hand-optimized agents use more environmental information than is available to the PSO trained neuro-controllers.

The neuro-controller with the highest fitness over all PSO iterations was compared to the hand-optimized agents using playing area sizes from 3×3 to 9×9 cells. Three measures were computed over 50 independent runs (each consisting of

100 games): the mean points scored ($\bar{p}$), the mean number of games won ($\bar{q}$), and the mean number of moves ($\bar{r}$). The standard deviations for the measures are respectively σ_p , σ_q , and σ_r . The statistical significance of performance differences between algorithms was assessed using the Mann-Whitney U test at a 95% confidence level with a Bonferroni correction.

TABLE I PARAMETERS FOR PSO TRAINED NEURO-CONTROLLERS

Parameter	Symbol	Value
Input neurons	m	10
Hidden neurons	n	10
Output neurons	o	4
Particle dimensionality	dim	154
Swarm size	$size$	100
Inertia weight	ξ	0.72
Cognitive and social terms	c_1 and c_2	1.42

```

Set a zero-valued counter for cells 2, 4, 6, and 8
if cell 1 is an obstacle or cell 8 is an obstacle then
  Increment counter for cell 2
end
if cell 2 is an obstacle or cell 3 is an obstacle then
  Increment counter for cell 4
end
if cell 4 is an obstacle or cell 5 is an obstacle then
  Increment counter for cell 6
end
if cell 6 is an obstacle or cell 7 is an obstacle then
  Increment counter for cell 8
end
Increment counters of cells that will move the agent closer to food
Move to non-obstacle cell with highest counter, breaking ties randomly

```

Fig. 4. Algorithm for the second hand-optimized agent.

B. Analysis of Results

Tables II to IV present empirical results. The p -value of a Mann-Whitney U test is set in bold whenever a statistically significant performance difference was identified.

Table II shows that the proposed method had better point scores than the hand-optimized agents for all playing area sizes. Larger areas showed a greater score differences.

Considering the games won (see Table III), the neuro-controllers far outperformed the simple agents on all playing area sizes except 7×7 and 9×9 , where no games were won on the largest playing area. The performance difference for 8×8 areas was also quite small. The poor large board performance of the neuro-controllers is possibly due to insufficient training time. The neuro-controllers performed fairly poorly in 5×5 and 7×7 spaces, probably because no safe movement patterns exist for spaces with an odd row and column count.

Table IV shows that the neuro-controllers always produced more moves than agents A_2 and A_3 . Interestingly, agent A_1 always produced more moves than the neuro-controllers, probably because it only aims to avoid collisions.

TABLE II
COMPARISON OF MEAN POINTS SCORED BY THE TECHNIQUES

Grid	PSO-NN		Simple Agents			p-value
	$\bar{p}$	σ_p	Agent	$\bar{p}$	σ_p	
3 × 3	7.961	0.142	A ₁	5.887	0.190	7.945×10^{-19}
			A ₂	6.550	0.152	7.933×10^{-19}
			A ₂	6.778	0.162	8.437×10^{-19}
4 × 4	13.702	1.131	A ₁	6.495	0.257	7.039×10^{-18}
			A ₂	8.625	0.287	7.028×10^{-18}
			A ₂	9.834	0.292	4.190×10^{-17}
5 × 5	18.094	1.790	A ₁	6.066	0.254	7.052×10^{-18}
			A ₂	10.379	0.368	7.055×10^{-18}
			A ₂	11.739	0.449	7.053×10^{-18}
6 × 6	22.040	3.778	A ₁	5.941	0.192	7.039×10^{-18}
			A ₂	11.550	0.376	7.050×10^{-18}
			A ₂	12.767	0.402	7.034×10^{-18}
7 × 7	25.141	5.236	A ₁	5.754	0.236	7.045×10^{-18}
			A ₂	12.806	0.507	9.788×10^{-17}
			A ₂	13.435	0.453	1.346×10^{-16}
8 × 8	29.699	6.545	A ₁	5.549	0.206	7.028×10^{-18}
			A ₂	14.029	0.470	7.057×10^{-18}
			A ₂	13.588	0.461	7.053×10^{-18}
9 × 9	32.161	5.654	A ₁	5.248	0.209	7.028×10^{-18}
			A ₂	14.974	0.428	5.630×10^{-17}
			A ₂	13.698	0.535	7.954×10^{-18}

TABLE III
COMPARISONS OF MEAN GAMES WON BY THE TECHNIQUES

Grid	PSO-NN		Simple Agents			p-value
	$\bar{q}$	σ_q	Agent	$\bar{q}$	σ_q	
3 × 3	97.240	8.826	A ₁	21.460	5.015	7.763×10^{-19}
			A ₂	30.080	4.597	7.692×10^{-19}
			A ₂	36.160	5.052	7.749×10^{-19}
4 × 4	64.920	22.700	A ₁	2.220	1.314	3.058×10^{-17}
			A ₂	2.620	1.563	7.006×10^{-17}
			A ₂	8.760	3.335	1.685×10^{-14}
5 × 5	5.720	6.452	A ₁	0.020	0.141	5.337×10^{-14}
			A ₂	0.000	0.000	2.127×10^{-14}
			A ₂	0.200	0.404	4.100×10^{-11}
6 × 6	10.620	21.305	A ₁	0.000	0.000	2.111×10^{-15}
			A ₂	0.000	0.000	2.111×10^{-15}
			A ₂	0.020	0.141	5.618×10^{-15}
7 × 7	0.060	0.314	A ₁	0.000	0.000	1.594×10^{-1}
			A ₂	0.000	0.000	1.594×10^{-1}
			A ₂	0.000	0.000	1.594×10^{-1}
8 × 8	1.220	4.220	A ₁	0.000	0.000	3.462×10^{-3}
			A ₂	0.000	0.000	3.462×10^{-3}
			A ₂	0.000	0.000	3.462×10^{-3}
9 × 9	0.000	0.000	A ₁	0.000	0.000	N/A
			A ₂	0.000	0.000	N/A
			A ₂	0.000	0.000	N/A

V. CONCLUSION

This paper proposed a novel method for training neuro-controllers using PSO, in order to play Snake. Existing Snake playing methods were described, each of which are incomparable to the new method. The performance of the proposed technique was thus compared to hand-optimized agents with the same sensory input as the new method. The neuro-controller method clearly outperformed the hand-optimized agents, demonstrating the approach's feasibility.

Future work will aim to improve the proposed method, particularly for larger playing areas. Different parameters, varying stopping conditions, and performance over training will be investigated. Comparisons to additional AI-based methods will be performed, possibly using variants of the methods discussed in Section II that use the same features as the novel method.

TABLE IV
COMPARISON OF MEAN MOVES MADE BY THE TECHNIQUES

Grid	PSO-NN		Simple Agents			p-value
	$\bar{r}$	σ_r	Agent	$\bar{r}$	σ_r	
3 × 3	24.072	1.722	A ₁	36.067	1.632	7.050×10^{-18}
			A ₂	17.037	0.570	7.047×10^{-18}
			A ₂	17.409	0.551	7.951×10^{-18}
4 × 4	58.831	6.749	A ₁	87.910	4.287	7.064×10^{-18}
			A ₂	32.464	1.175	7.061×10^{-18}
			A ₂	41.708	1.331	2.704×10^{-15}
5 × 5	128.704	22.734	A ₁	155.604	7.987	1.443×10^{-10}
			A ₂	49.805	2.255	7.066×10^{-18}
			A ₂	76.798	3.190	9.526×10^{-17}
6 × 6	210.344	47.864	A ₁	238.996	11.098	1.263×10^{-7}
			A ₂	64.537	2.454	7.064×10^{-18}
			A ₂	117.514	4.046	2.197×10^{-17}
7 × 7	312.692	108.951	A ₁	350.627	16.807	8.317×10^{-6}
			A ₂	80.799	3.788	1.349×10^{-16}
			A ₂	158.839	5.353	1.312×10^{-15}
8 × 8	443.488	138.502	A ₁	469.400	21.495	8.713×10^{-3}
			A ₂	98.518	4.028	7.064×10^{-18}
			A ₂	199.778	7.068	8.108×10^{-15}
9 × 9	559.173	155.054	A ₁	598.626	28.272	1.689×10^{-3}
			A ₂	114.007	3.679	1.349×10^{-16}
			A ₂	239.083	9.915	1.430×10^{-16}

REFERENCES

- [1] B. Coppin, *Artificial Intelligence Illuminated*. Jones & Bartlett, 2004.
- [2] I. Millington and J. Funge, *Artificial Intelligence for Games*, 2nd ed. Morgan Kaufmann, 2009.
- [3] A. L. Samuel, "Some studies in machine learning using the game of checkers," *IBM J. Res. Develop.*, vol. 3, no. 3, pp. 210–229, 1959.
- [4] M. Campbell, A. J. Hoane, Jr., and F.-h. Hsu, "Deep blue," *Artif. Intell.*, vol. 134, no. 1–2, pp. 57–83, 2002.
- [5] D. Silver *et al.*, "Mastering the game of Go with deep neural networks and tree search," *Nature*, vol. 529, pp. 484–489, 2016.
- [6] R. DeMaria and J. L. Wilson, *High Score!: The Illustrated History of Electronic Games*, 2nd ed. McGraw-Hill, 2002.
- [7] G. Osborn, "Always on the move: A history of mobile gaming," 2014.
- [8] H. Bodlaender and T. Kloks, "Fast algorithms for the tron game on trees," Utrecht Univ., Tech. Rep. RUU-CS-90-11, 1990.
- [9] P. Funes, "Evolution of complexity in real-world domains," Ph.D. dissertation, Brandeis Univ., 2001.
- [10] S. Samothrakakis, D. Robles, and S. M. Lucas, "A UCT agent for Tron: Initial investigations," in *Proc. IEEE CIG*, 2010, pp. 365–371.
- [11] T. Ehrlis, "Application of genetic programming to the snake game," in *Advanced Game Programming: A GameDev.net Collection*, J. Hattan and D. Sikora, Eds. Cengage, 2009, pp. 113–136.
- [12] C. Lockhart, "Application of temporal difference learning to the game of Snake," Master's thesis, Univ. Louisville, 2010.
- [13] R. S. Sutton, "Learning to predict by the methods of temporal differences," *Mach. Learn.*, vol. 3, no. 1, pp. 9–44, 1988.
- [14] J.-F. Yeh, P.-H. Su, S.-H. Huang, and T.-C. Chiang, "Snake game AI: Movement rating functions and evolutionary algorithm-based optimization," in *Proc. TAAI*, 2016, pp. 256–261.
- [15] C. M. Bishop, *Neural Networks for Pattern Recognition*. Oxford Univ. Press, 1995.
- [16] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proc. IEEE ICNN*, 1995, pp. 1942–1948.
- [17] L. F. A. Wessels and E. Barnard, "Avoiding false local initialization of minima by proper connections," *IEEE Trans. Neural Netw.*, vol. 3, no. 6, pp. 899–905, 1992.
- [18] Y. Shi and R. Eberhart, "A modified particle swarm optimizer," in *Proc. IEEE CEC*, 1998, pp. 69–73.
- [19] M. Clerc and J. Kennedy, "The particle swarm—explosion, stability, and convergence in a multidimensional complex space," *IEEE Trans. Evol. Comput.*, vol. 6, no. 1, pp. 58–73, 2002.
- [20] N. J. Nilsson, *Artificial Intelligence: A New Synthesis*. Morgan Kaufmann, 1998.